

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Ульяновский государственный технический университет»
Кафедра «Вычислительная техника»

Операционные системы

(название дисциплины)

Лабораторная работа №3

Менеджер виртуальной памяти

(название работы)

Выполнил

Студент группы ИВТИИБД-21

Мокшин Д.И.

(Фамилия И.О.)

Проверил(а):

ст. преподаватель кафедры «ВТ»

(должность)

Беляев К.С.

(Фамилия И.О.)

СОДЕРЖАНИЕ

1. Задание на лабораторную работу.....	3
2. Описание реализации.....	3
3. Тестирование.....	6
На рисунках 3.1-3.2 представлены тест программы.....	7
Рис. 3.1. Тест программы.....	7
Рис. 3.2. статистика.....	7
4. Вывод.....	7
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ.....	8

1. Задание на лабораторную работу

Необходимо разработать программу на языке C, моделирующую преобразование логических адресов в физические для виртуального адресного пространства. Программа должна считывать файл, содержащий 32-битные целые числа, представляющие логические адреса. Требуется реализовать таблицу страниц, TLB. Программа должна обрабатывать ошибки страниц с использованием BACKING_STORE.bin. Для каждого логического адреса из входного файла программа должна вывести сам логический адрес, преобразованный физический адрес и значение знакового байта, хранящееся по этому физическому адресу.

2. Описание реализации

Для выполнения лабораторной работы была разработана программа на языке C, реализующая упрощенный менеджер виртуальной памяти.

Программа моделирует механизм виртуальной памяти с использованием страничной организации адресов. В ней определены основные константы: размер страницы составляет 256 байт, количество страниц и фреймов равно 256, размер TLB — 16 записей, а общий объем физической памяти — 64 КБ.

Для работы с TLB используется структура TLBEntry рис2.1, которая содержит номер страницы, номер фрейма, флаг валидности и счетчик LRU для реализации политики замещения.

```
typedef struct {  
    int page_number;  
    int frame_number;  
    int valid;  
    int lru_counter;  
} TLBEntry;
```

Рис 2.1 TLBEntry структура

Таблица страниц представлена массивом page_table, а физическая память — массивом physical_memory Рис 2.2. Также в программе ведется

статистика: общее количество обработанных адресов, количество ошибок страниц (page faults) и количество попаданий в TLB.

```
int page_table[PAGE_TABLE_SIZE];
char physical_memory[MEMORY_SIZE];
```

Рис 2.3 массивы

Инициализация системы осуществляется в функции `init_system`, где обнуляется таблица страниц, сбрасываются все записи TLB и очищается физическая память. Поиск записи в TLB выполняет функция `search_tlb` Рис2.4. При обнаружении нужной страницы она обновляет счетчик LRU и возвращает номер фрейма.

```
int search_tlb(int page_number) {
    for (int i = 0; i < TLB_SIZE; i++) {
        if (tlb[i].valid && tlb[i].page_number == page_number) {
            tlb[i].lru_counter = lru_clock++;
            return tlb[i].frame_number;
        }
    }
    return -1;
}
```

Рис 2.4

Если страница в TLB не найдена, функция `update_tlb` Рис 2.5 добавляет новую запись. При отсутствии свободных мест в TLB заменяется запись с наименьшим значением счетчика LRU.

```
void update_tlb(int page_number, int frame_number) {
    int free_slot = -1;
    for (int i = 0; i < TLB_SIZE; i++) {
        if (!tlb[i].valid) {
            free_slot = i;
            break;
        }
    }

    if (free_slot == -1) {
        int lru_index = 0;
        int min_counter = tlb[0].lru_counter;
        for (int i = 1; i < TLB_SIZE; i++) {
            if (tlb[i].lru_counter < min_counter) {
                min_counter = tlb[i].lru_counter;
                lru_index = i;
            }
        }
        free_slot = lru_index;
    }

    tlb[free_slot].page_number = page_number;
    tlb[free_slot].frame_number = frame_number;
    tlb[free_slot].valid = 1;
    tlb[free_slot].lru_counter = lru_clock++;
}
```

Рис 2.5 update_tlb

Обработка ошибки страницы реализована в функции `handle_page_fault` Рис 2.6. При возникновении `page fault` программа с помощью `lseek` перемещается к нужной позиции в файле `BACKING_STORE.bin`, считывает целиком одну страницу размером 256 байт с помощью `read` и загружает её в очередной свободный фрейм физической памяти. После этого обновляется таблица страниц и увеличивается счетчик `page_faults`.

```
int handle_page_fault(int page_number) {
    if (free_frame_index >= FRAME_COUNT) {
        fprintf(stderr, "Error: Physical memory full (should not happen with 256 frames)\n");
        exit(1);
    }

    off_t offset = page_number * PAGE_SIZE;

    if (lseek(backing_fd, offset, SEEK_SET) < 0) {
        perror("lseek error");
        exit(1);
    }

    char *frame_ptr = &physical_memory[free_frame_index * PAGE_SIZE];
    ssize_t bytes_read = read(backing_fd, frame_ptr, PAGE_SIZE);
    if (bytes_read != PAGE_SIZE) {
        fprintf(stderr, "Error reading page %d from backing store\n", page_number);
        exit(1);
    }
    page_table[page_number] = free_frame_index;

    int allocated_frame = free_frame_index;
    free_frame_index++;

    page_faults++;
    return allocated_frame;
}
```

Рис 2.6 `handle_page_fault`

Основная логика трансляции логического адреса в физический находится в функции `get_physical_address` Рис 2.7. Сначала выполняется поиск страницы в TLB. В случае попадания увеличивается счетчик `tlb_hits`. При промахе проверяется таблица страниц. Если страница ещё не загружена в физическую память, вызывается функция `handle_page_fault`. После получения номера фрейма запись добавляется в TLB с помощью `update_tlb`. В конце физический адрес формируется путём добавления смещения (`offset`) к базовому адресу фрейма.

В функции main программа открывает файл addresses.txt для чтения логических адресов и файл BACKING_STORE.bin для доступа к содержимому страниц. Для каждого прочитанного логического адреса выполняется его трансляция в физический адрес, считывается значение байта по полученному адресу и выводится на экран в формате «Logical Physical Value».

После обработки всех адресов выводится итоговая статистика работы системы, включающая общее количество обработанных адресов, количество и процент ошибок страниц, а также количество и процент попаданий в TLB.

3. Тестирование

На рисунках 3.1-3.2 представлены тест программы.

```
Logical: 40185 -> Physical: 1273 -> Value: 0
Logical: 28781 -> Physical: 1389 -> Value: 0
Logical: 24462 -> Physical: 1678 -> Value: 23
Logical: 48399 -> Physical: 1807 -> Value: 67
Logical: 64815 -> Physical: 2095 -> Value: 75
Logical: 18295 -> Physical: 2423 -> Value: -35
Logical: 12218 -> Physical: 2746 -> Value: 11
Logical: 22760 -> Physical: 3048 -> Value: 0
Logical: 57982 -> Physical: 3198 -> Value: 56
Logical: 27966 -> Physical: 3390 -> Value: 27
Logical: 54894 -> Physical: 3694 -> Value: 53
Logical: 38929 -> Physical: 3857 -> Value: 0
Logical: 32865 -> Physical: 4193 -> Value: 0
Logical: 64243 -> Physical: 4595 -> Value: -68
Logical: 2315 -> Physical: 4619 -> Value: 66
Logical: 64454 -> Physical: 5062 -> Value: 62
Logical: 55041 -> Physical: 5121 -> Value: 0
Logical: 18633 -> Physical: 5577 -> Value: 0
Logical: 14557 -> Physical: 5853 -> Value: 0
Logical: 61006 -> Physical: 5966 -> Value: 59
Logical: 62615 -> Physical: 407 -> Value: 37
Logical: 7591 -> Physical: 6311 -> Value: 105
Logical: 64747 -> Physical: 6635 -> Value: 58
Logical: 6727 -> Physical: 6727 -> Value: -111
Logical: 32315 -> Physical: 6971 -> Value: -114
Logical: 60645 -> Physical: 7397 -> Value: 0
Logical: 6308 -> Physical: 7588 -> Value: 0
Logical: 45688 -> Physical: 7800 -> Value: 0
Logical: 969 -> Physical: 8137 -> Value: 0
Logical: 40891 -> Physical: 8379 -> Value: -18
Logical: 49294 -> Physical: 8590 -> Value: 48
Logical: 41118 -> Physical: 8862 -> Value: 40
Logical: 21395 -> Physical: 9107 -> Value: -28
Logical: 6091 -> Physical: 9419 -> Value: -14
Logical: 32541 -> Physical: 9501 -> Value: 0
Logical: 17665 -> Physical: 9729 -> Value: 0
Logical: 3784 -> Physical: 10184 -> Value: 0
Logical: 28718 -> Physical: 1326 -> Value: 28
Logical: 59240 -> Physical: 10344 -> Value: 0
Logical: 40178 -> Physical: 1266 -> Value: 39
Logical: 60086 -> Physical: 10678 -> Value: 58
Logical: 42252 -> Physical: 10764 -> Value: 0
Logical: 44770 -> Physical: 11234 -> Value: 43
Logical: 22514 -> Physical: 11506 -> Value: 21
Logical: 3067 -> Physical: 11771 -> Value: -2
```

Рис. 3.1. Тест программы

```
Всего обработано: 1000
ошибок страниц: 244
частота ошибок страниц: 24.40%
частота TLB: 55
Частота попаданий в TLB: 5.50%
danil@WIN-CUBCPHMT5SL:~/lab3$
```

Рис. 3.2. статистика

4. Вывод

В ходе лабораторной работы на языке C реализован симулятор менеджера виртуальной памяти. Программное решение осуществляет трансляцию 16-битных логических адресов в физические, используя структуру «8 бит на номер страницы / 8 бит на смещение». Архитектура включает таблицу страниц (256 записей) и буфер ассоциативной трансляции (TLB) на 16 записей с алгоритмом вытеснения LRU. Механизм подкачки реализован путем считывания данных из внешнего бинарного файла BACKING_STORE.bin.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Керниган Б. У. Язык программирования С / Б. У. Керниган, Д. М. Ричи.
- 2-е изд. - М. : Вильямс, 2021. - 304 с.
2. Робачевский А. М. Операционная система UNIX / А. М. Робачевский. -
2-е изд. - СПб. : БХВ-Петербург, 2010. - 656 с.